MANIPAL UNIVERSITY JAIPUR

(University under Section 2(f) of the UGC Act)

Cricket Tournament Management System (CricketBuddy)

PROJECT REPORT

Submitted in partial fulfillment of the requirements for the laboratory course of

Relational Database Management Systems Laboratory (CS2102)

Submitted By:

Raunak Kwatra
Reg. No: 2502050081

Kartikey Aggarwal
Reg. No: 2502050082

Under the Supervision of:

Dr. Onkar Singh
Assistant Professor (Senior Scale)
Department of CSE

Department of Computer Science and Engineering

Manipal University Jaipur

Dehmi Kalan, Near GVK Toll Plaza, Jaipur-Ajmer Expressway, Jaipur, Rajasthan 303007

B.Tech Computer Science and Engineering — 3rd Semester

Academic Session: 2026–2027

Certificate of Recommendation

This is to certify that the project report entitled "Cricket Tournament Management System (CricketBuddy)" submitted by Raunak Kwatra (Registration No: 2502050081) and Kartikey Aggarwal (Registration No: 2502050082) in partial fulfillment of the academic requirements for the Relational Database Management Systems Laboratory (CS2102) of Bachelor of Technology in Computer Science and Engineering at Manipal University Jaipur, is a bonafide record of the design, normalization, development, and testing work carried out under my direct supervision.

The database schema design, normalized relations (satisfying Boyce-Codd Normal Form), Express REST API backend, and responsive frontend interface presented in this report have reached the standard of academic and engineering competence required by the Department.

To the best of my knowledge, the matter embodied in this report has not been submitted to any other University or Institution for the award of any degree or diploma.

Dr. Onkar Singh
Project Supervisor
Department of CSE, MUJ

Head of Department
Department of Computer Science & Engg.
Manipal University Jaipur

Candidate's Declaration

We hereby declare that the project report entitled "Cricket Tournament Management System (CricketBuddy)" submitted to the Department of Computer Science and Engineering, Manipal University Jaipur, is an authentic record of our own research and software engineering work carried out during the Autumn 2026 semester.

We confirm that:

1. The work has been done by us independently under the mentorship of our faculty supervisor, Dr. Onkar Singh.

2. All sources of code, theoretical references, textbooks, and documentation manuals have been duly acknowledged.

3. The schema and application code have not formed the basis for the award of any other degree, diploma, or fellowship at this or any other institution.

Raunak Kwatra
Registration No: 2502050081

Kartikey Aggarwal
Registration No: 2502050082

Acknowledgement

We express our profound gratitude to our project supervisor, Dr. Onkar Singh, Assistant Professor (Senior Scale), Department of Computer Science and Engineering, Manipal University Jaipur, for his invaluable guidance, technical insights, and continuous encouragement throughout the conception, modeling, and realization of the Cricket Tournament Management System.

We are deeply thankful to the Head of Department, Computer Science and Engineering, for providing world-class laboratory facilities, database infrastructure, and a conducive academic environment.

We extend our heartfelt appreciation to our peers, families, and friends whose unwavering moral support and feedback enabled us to complete this project successfully.

Abstract

The Cricket Tournament Management System (CricketBuddy) is an enterprise-grade, database-driven web application architected to streamline and modernize the administrative workflows of professional cricket leagues, championships, and collegiate tournaments. The principal objective of this system is to deliver a centralized, secure, and interactive management portal coordinating nine critical operational entities: franchise teams, stadiums and host venues, squad players, certified match umpires, tournament fixtures, commercial sponsorships, match scorecards, team participations, and officiating referees.

The foundational layer uses MySQL 9.7 as the Relational Database Management System (RDBMS) under the database catalog CricketTournamentDB. The database schema has been engineered strictly adhering to relational algebra, third normal form (3NF), and Boyce-Codd normal form (BCNF). It establishes primary keys, composite keys, cascading foreign key constraints, and integrity rules that guarantee consistency and eliminate deletion and update anomalies across tournament operations.

The service layer is powered by Node.js and Express.js, providing an asynchronous RESTful API architecture connecting client interactions to the MySQL database engine through an optimized connection pool. The presentation layer is built with modern HTML5, CSS3, dynamic JavaScript, and Lucide icons, offering a responsive user interface with live global search, dynamic statistical filtering, theme toggle (dark/light), and an intuitive dual-mode role-based access control (RBAC). In this architecture, administrators authenticate via a secure master password to unlock complete CRUD capabilities, while viewers enter their identity details to browse rosters, fixtures, valuations, and match scorecards in a safe read-only environment.

This project demonstrates practical mastery over relational database principles including Entity-Relationship (ER) modeling, multi-table JOINs, aggregation functions, subqueries, referential integrity cascading, and full-stack API integration.

Keywords: Relational Database Management System (RDBMS), MySQL 9.7, Boyce-Codd Normal Form (BCNF), Entity-Relationship (ER) Modeling, RESTful API, Express.js, Cricket Tournament Administration, Role-Based Access Control (RBAC).

Table of Contents

Abstract ..... **iv**
List of Figures ..... **vi**
List of Tables ..... **vii**
Chapter 1: Introduction **1**
1.1 Project Overview 1
1.2 Objectives of the Project 2
1.3 Importance and Relevance 2
1.4 Scope and Limitations 3
Chapter 2: Literature Review **4**
2.1 Existing Cricket Tournament Systems 4
2.2 Relevant Technologies 4
2.2.1 MySQL Relational Database Management System 4
2.2.2 Node.js and Express.js REST API Layer 5
2.2.3 Vanilla JavaScript, HTML5, CSS3 and Lucide Icons 5
2.3 Summary 6
Chapter 3: System Design **7**
3.1 Database Design Overview 7
3.2 Entity-Relationship (ER) Diagram 7
3.3 Description of Tables and Attributes 8
3.4 Database Normalization 12
3.5 Limitations of Normalization 14
3.6 Database Views (Virtual Tables) 14
Chapter 4: Implementation and Screenshots **15**
4.1 Overview 15
4.2 Database Implementation 15
4.3 Tournament Overview Dashboard 17
4.4 Team Management and Franchises 18
4.5 Player Squad Roster and Athlete Profiles 19
4.6 Tournament Matches and Fixture Scheduling 20
4.7 Match Scorecards and Performance Logs 21
4.8 Stadiums and Host Venues 22
4.9 Tournament Analytics and Aggregated Queries 23
4.10 Commercial Team Sponsorships 24
4.11 Certified Match Umpires and Referees 25
4.12 Dual-Mode Authentication and Role-Based Access Control 26
4.13 Live Global Search and Dropdown Filtering 27
Chapter 5: Results and Discussion **28**
5.1 Presentation of Results 28
5.2 Discussion of Findings 28
5.3 Testing and Validation 29
Chapter 6: Conclusion **31**
6.1 Summary of Project Work 31
6.2 Achievement of Project Objectives 31

6.3 Lessons Learned 32
Chapter 7: Future Work 33
7.1 Suggestions for Future Enhancements 33
7.2 Areas for Further Research 33
References 34

List of Figures

Figure 3.1: Entity-Relationship (ER) Diagram for CricketTournamentDB 8
Figure 4.1: MySQL Terminal Verification — SHOW TABLES in CricketTournamentDB 15
Figure 4.2: MySQL Schema Structure — DESCRIBE CricketTeam and CricketPlayer 16
Figure 4.3: CricketBuddy Tournament Executive Dashboard & KPI Metrics 17
Figure 4.4: Team Management Roster & Franchise Profiles 18
Figure 4.5: Player Squad Roster with Role Badges & Team Affiliations 19
Figure 4.6: Tournament Matches & Fixture Scheduling Module 20
Figure 4.7: Match Scorecards & Batting/Bowling Performance Tracking 21
Figure 4.8: Stadiums and Host Venues Management Interface 22
Figure 4.9: Tournament Analytics & Queries (Top Run Scorers, Squad Sizes, Sponsorships) 23
Figure 4.10: Commercial Team Sponsorships & Contract Valuations View 24
Figure 4.11: Certified Match Referees and Umpires Directory 25
Figure 4.12: Dual-Mode Role-Based Authentication Modal (Admin vs Viewer) 26
Figure 4.13: Live Global Search & Real-Time Dropdown Filtering 27

List of Tables

Table 3.1: CricketTeam Table Schema 9
Table 3.2: CricketStadium Table Schema 9
Table 3.3: CricketPlayer Table Schema 10
Table 3.4: CricketUmpire Table Schema 10
Table 3.5: TournamentMatch Table Schema 11
Table 3.6: TeamSponsor Table Schema 11
Table 3.7: MatchScorecard Table Schema 12
Table 3.8: TeamParticipation Table Schema 12
Table 3.9: MatchOfficials Table Schema 12
Table 3.10: Catalog of Database Views in CricketTournamentDB 14

Chapter 1: Introduction

1.1 Project Overview

Cricket is celebrated as one of the most widely followed professional sports globally, engaging hundreds of franchises, thousands of athletic players, multimillion-dollar corporate sponsorships, and millions of passionate spectators. Organizing modern cricket tournaments—whether multi-tier domestic championships, premier leagues, or collegiate tournaments—demands the systematic coordination of interlinked operational data, including participating teams, player rosters, certified umpires, venues, match schedules, live scorecards, and commercial sponsorship contracts governed under regulations modeled on the International Cricket Council (ICC) standards [1].

Traditionally, tournament management in universities and local athletic boards relied upon decentralized spreadsheets, paper scorebooks, and disconnected manual record-keeping. These antiquated methodologies suffer from severe data redundancies, lack of concurrency control, arithmetic inconsistencies in player averages, loss of historical records, and security vulnerabilities.

To eliminate these challenges, the Cricket Tournament Management System (CricketBuddy) was conceived as a centralized, relational database-driven web platform following robust database design principles [2]. Developed using MySQL 9.7 as the primary RDBMS, an Express.js / Node.js backend server, and a high-performance interactive frontend interface, CricketBuddy provides automated record management, referential integrity, and transparent data exploration for both administrators and viewers.

Source Code & Repository:

The complete source code, normalized SQL schema definitions, backend REST API routes, frontend controllers, and deployment configuration are maintained on GitHub:

GitHub Repository:https://github.com/raunakkwatra/CricketBuddy

1.2 Objectives of the Project

The primary objectives established for the CricketBuddy system comprise:

- To architect an authentic relational database schema normalized to BCNF under CricketTournamentDB, capturing 9 interrelated entities without data anomalies [2], [3].
- To design an expressive Chen-notation Entity-Relationship (ER) model identifying cardinality ratios (1:N, M:N) and primary/foreign key mappings [3].
- To enforce database integrity through primary keys, composite keys, NOT NULL rules, CHECK constraints, and cascading foreign keys (ON DELETE CASCADE, ON DELETE SET NULL).
- To build an asynchronous Node.js and Express.js REST API implementing parameterized CRUD operations across all tournament modules.
- To construct a dual-mode role-based access control (RBAC) interface allowing administrators to manage records and viewers to browse data in read-only mode.
- To deliver live global search filtering, theme toggling (dark/light), and analytical aggregations (top run-scorers, squad counts, commercial valuations).

1.3 Importance and Relevance

Managing a multi-franchise sports tournament involves complex financial and athletic logistics. High-precision financial contracts (e.g., Tata Steel $25,000,000,000.00 sponsorship) must be tracked alongside individual player batting strike rates and bowling figures without arithmetic inaccuracies. A normalized relational database guarantees that modifying a team's coach or stadium automatically reflects across all associated matches and rosters, eliminating manual re-entry.

1.4 Scope and Limitations

The current implementation encompasses a functional local tournament management suite running on MySQL 9.7 and Node.js. Limitations include:

- Offline / Single-Host Deployment: The current build is configured for local intranet hosting (localhost:3000) rather than a geo-distributed cloud cluster.
- Match Simulation: Match outcomes are currently entered by certified umpires/administrators rather than pulled from external IoT ball-tracking radars.
- Payment Gateway: Commercial sponsorship deals are recorded as ledger entries rather than integrated with live bank payment gateways.

Chapter 2: Literature Review

2.1 Existing Cricket Tournament Systems

Modern sports administration has shifted rapidly from manual paperwork to integrated digital scorekeeping systems such as Cricbuzz, ESPNcricinfo, and CricHeroes. While these commercial services offer vast international coverage, they operate as proprietary walled gardens unsuitable for collegiate athletic boards, regional leagues, and university tournaments that require custom governance rules, sponsorship ledgers, and venue allocation tools.

2.2 Relevant Technologies

2.2.1 MySQL Relational Database Management System

MySQL is an industry-leading open-source RDBMS renowned for ACID compliance (Atomicity, Consistency, Isolation, Durability), reliable InnoDB storage engine, multi-version concurrency control (MVCC), and structured query optimization [4]. In CricketBuddy, MySQL 9.7 manages the 9 core relations, indexing primary and foreign keys for instantaneous join lookups [4].

2.2.2 Node.js and Express.js REST API Layer

Node.js provides an asynchronous, event-driven JavaScript runtime ideal for I/O-intensive database operations [5]. Express.js acts as the server framework [6], exposing REST endpoints (/api/teams, /api/players, /api/stadiums, /api/matches, /api/scorecards, /api/sponsors, /api/umpires, /api/statistics). Client-server communication strictly follows stateless RESTful architectural constraints and standard HTTP verbs (GET, POST, PUT, DELETE) with structured JSON payloads [7]. The service layer utilizes the mysql2 connection pool with parameterized queries to prevent SQL injection.

2.2.3 Vanilla JavaScript, HTML5, CSS3 and Lucide Icons

The presentation tier is engineered using vanilla ES6+ JavaScript, semantic HTML5, modern CSS3 specifications [8] with custom variables, and offline-bundled Lucide SVG icons. This ensures sub-50ms DOM rendering, zero build-step overhead, and smooth responsiveness across desktop and mobile devices.

2.3 Summary

The integration of a normalized MySQL backend with an Express REST API and a responsive presentation frontend delivers a resilient, secure, and intuitive tournament governance suite that addresses the shortcomings of legacy manual scorekeeping.

Chapter 3: System Design

3.1 Database Design Overview

The design of CricketTournamentDB follows standard relational database lifecycle principles as articulated by Silberschatz et al. [2]: conceptual entity identification, cardinality mapping, relational schema decomposition, and BCNF verification. The system represents 9 entities linked via 1-to-many and many-to-many relationships.

3.2 Entity-Relationship (ER) Diagram

Figure 3.1 depicts the formal Entity-Relationship diagram in Chen's notation, adhering to standard ER modeling conventions described by Elmasri and Navathe [3]. Entities are shown as rectangles, attributes as ovals (primary keys underlined), relationships as diamonds, and associative tables as double diamonds.

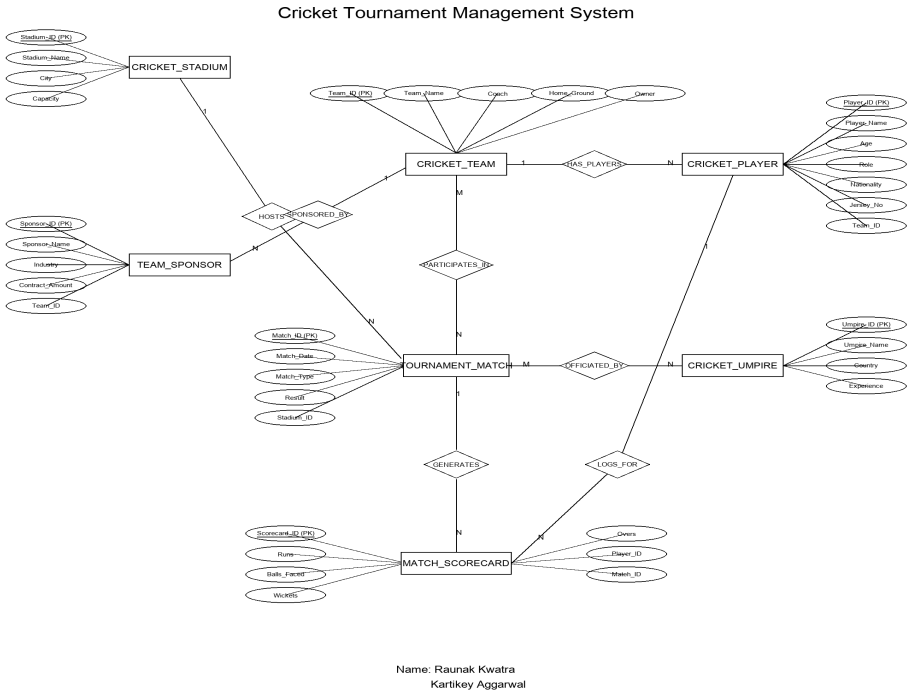


Figure 3.1: Entity-Relationship (ER) Diagram for CricketTournamentDB (Chen's Notation)

3.3 Description of Tables and Attributes

The physical relational schema comprises nine normalized tables:

Table 3.1: CricketTeam Table Schema

Field	Type	Key	Description
TeamID	INT	PRI	Unique team identifier; auto-incremented primary key.

Field	Type	Key	Description
TeamName	VARCHAR(100)	UNI	Official franchise name; unique and mandatory.
Coach	VARCHAR(100)		Head coach name overseeing athletic strategy.
HomeGround	VARCHAR(100)		Designated home venue of the franchise.
Owner	VARCHAR(100)		Corporate owner or franchise consortium.

Table 3.2: CricketStadium Table Schema

Field	Type	Key	Description
StadiumID	INT	PRI	Unique venue identifier; auto-incremented primary key.
StadiumName	VARCHAR(100)	UNI	Official stadium name; mandatory.
City	VARCHAR(50)		Geographic host city location.
Capacity	INT		Spectator seating capacity (must be positive).

Table 3.3: CricketPlayer Table Schema

Field	Type	Key	Description
PlayerID	INT	PRI	Unique athlete identifier; auto-incremented primary key.
PlayerName	VARCHAR(100)		Full name of the registered player.
Age	INT		Athlete age in years (must be between 15 and 50).
Role	VARCHAR(50)		Specialization: Batsman, Bowler, All Rounder, Wicket Keeper.
Nationality	VARCHAR(50)		Country of international citizenship.
JerseyNo	INT		Official jersey squad number.
TeamID	INT	FK	Foreign key linking to CricketTeam(TeamID); ON DELETE SET NULL.

Table 3.4: CricketUmpire Table Schema

Field	Type	Key	Description
UmpireID	INT	PRI	Unique umpire identifier; auto-incremented primary key.
UmpireName	VARCHAR(100)		Full name of certified match referee.
Country	VARCHAR(50)		Country of officiating board accreditation.
Experience	INT		Years of certified professional umpiring experience.

Table 3.5: TournamentMatch Table Schema

Field	Type	Key	Description
MatchID	INT	PRI	Unique fixture identifier; auto-incremented primary key.
MatchDate	DATE		Scheduled calendar date of match fixture.
MatchType	VARCHAR(50)		Match format: T20, ODI, or Test.
StadiumID	INT	FK	Foreign key references CricketStadium(StadiumID); ON DELETE SET NULL.
Result	VARCHAR(100)		Official outcome or current status of fixture.

Table 3.6: TeamSponsor Table Schema

Field	Type	Key	Description
SponsorID	INT	PRI	Unique sponsorship deal identifier; primary key.
SponsorName	VARCHAR(100)		Corporate brand or company sponsor name.
Industry	VARCHAR(50)		Commercial market sector (e.g., Conglomerate, Tech).
ContractAmount	DECIMAL(18,2)		High-precision monetary contract valuation.
TeamID	INT	FK	Foreign key references CricketTeam(TeamID); ON DELETE CASCADE.

Table 3.7: MatchScorecard Table Schema

Field	Type	Key	Description
ScorecardID	INT	PRI	Unique player-performance log identifier; primary key.
MatchID	INT	FK	Foreign key references TournamentMatch(MatchID); ON DELETE CASCADE.
PlayerID	INT	FK	Foreign key references CricketPlayer(PlayerID); ON DELETE CASCADE.
Runs	INT		Batting runs scored by player in this match fixture.
BallsFaced	INT		Total deliveries faced at the batting crease.
Wickets	INT		Bowled wickets claimed during the innings.
Overs	DECIMAL(4,1)		Total bowling overs delivered (e.g., 4.0).

Table 3.8: TeamParticipation Table Schema

Field	Type	Key	Description
TeamID	INT	PRI, FK	Composite PK part 1; references CricketTeam(TeamID).
MatchID	INT	PRI, FK	Composite PK part 2; references TournamentMatch(MatchID).

Table 3.9: MatchOfficials Table Schema

Field	Type	Key	Description
MatchID	INT	PRI, FK	Composite PK part 1; references TournamentMatch(MatchID).
UmpireID	INT	PRI, FK	Composite PK part 2; references CricketUmpire(UmpireID).

3.4 Database Normalization

Database normalization is the formal mathematical technique of organizing relations to minimize redundancy and eliminate insert, update, and delete anomalies [2]. The schema CricketTournamentDB satisfies Boyce-Codd Normal Form (BCNF):

1. First Normal Form (1NF)

Requirement: All domain attributes must hold strictly atomic scalar values without repeating groups or nested arrays.

Verification: In all nine tables, every attribute holds scalar data. Multi-valued elements (such as multiple players on a team or multiple umpires per match) are modeled as distinct atomic rows across associative junction tables (TeamParticipation, MatchOfficials). Hence, 1NF is strictly met.

2. Second Normal Form (2NF)

Requirement: The relation must be in 1NF and every non-prime attribute must be fully functionally dependent on the entire candidate key (no partial key dependencies).

Verification: In the single-key tables (CricketTeam, CricketStadium, CricketPlayer, CricketUmpire, TournamentMatch, TeamSponsor, MatchScorecard), the primary key is atomic, making partial dependency impossible. In the composite junction tables (TeamParticipation, MatchOfficials), all attributes form the composite key with zero non-prime attributes. Hence, 2NF is strictly met.

3. Third Normal Form (3NF)

Requirement: The relation must be in 2NF and no non-prime attribute can be transitively dependent on the primary key (no X -> Y -> Z dependencies).

Verification: In CricketPlayer, team attributes (Coach, HomeGround) are not duplicated; only TeamID is stored. In TournamentMatch, stadium details (City, Capacity) are omitted and referenced solely through StadiumID. Non-prime attributes depend solely and directly on their relation's primary key. Hence, 3NF is strictly met.

4. Boyce-Codd Normal Form (BCNF)

Requirement: As formalized by Elmasri and Navathe [3], for every non-trivial functional dependency X -> Y, X must be a superkey.

Verification: Across all tables, all determinants are candidate keys or primary keys. There are no overlapping candidate keys or anomalies where a non-prime attribute determines part of a key. Thus, BCNF is fully satisfied.

3.5 Limitations of Normalization

While high normalization guarantees data consistency, it involves practical trade-offs:

- Multi-Table JOIN Overhead: Reconstructing complete match scorecards requires joining five tables (MatchScorecard, CricketPlayer, CricketTeam, TournamentMatch, CricketStadium), which incurs query execution time at scale.
- Aggregation Complexity: Calculating tournament-wide leaderboards (top run-scorers, team run totals) demands real-time GROUP BY computations across normalized tables.

3.6 Database Views (Virtual Tables)

To resolve multi-table JOIN overhead while strictly preserving Boyce-Codd Normal Form (BCNF) in the foundational tables, seven relational database Views were designed and deployed directly in MySQL under CricketTournamentDB using the MySQL 9.7 database engine [4]. In relational database theory [2], a VIEW

acts as a pre-compiled virtual relation dynamically produced by the database query optimizer. This provides major system benefits:

1. Query Simplification & Abstraction: Client applications, REST API routes, and analytics dashboards query a single logical entity instead of authoring repetitive, multi-way JOIN clauses across five distinct tables.

2. Security and Logical Data Independence: Restricts access to sensitive base table columns while presenting customized operational projections to authenticated viewers and external consumers.

3. Uniformity: Centralizes relationship definitions so changes to underlying foreign keys do not invalidate application frontends.

Table 3.10: Catalog of Database Views in CricketTournamentDB

View Name	Base Relations Joined	Purpose & Encapsulated Logic
TeamDetails	CricketTeam	Encapsulates franchise metadata including Head Coach, Home Venue, and Corporate Owner.
PlayerDetails	CricketPlayer, CricketTeam	Performs LEFT JOIN to pair player biographical profiles with human-readable Team names.
MatchDetails	TournamentMatch, CricketStadium	Resolves match schedules and fixtures with physical Stadium names and host City locations.
TeamSponsorDetails	TeamSponsor, CricketTeam	Pairs corporate sponsor brands, commercial valuations, and industry types with sponsored Teams.
MatchScorecardDetails	MatchScorecard, CricketPlayer, CricketTeam, TournamentMatch	Multi-table JOIN connecting player names, team affiliations, match dates, runs scored, balls, and wickets.
TeamMatchParticipation	TeamParticipation, CricketTeam, TournamentMatch	Consolidates fixture participation records with team names, match dates, formats, and results.
MatchOfficialsDetails	MatchOfficials, TournamentMatch, CricketUmpire	Pairs match fixtures with accredited umpires, nationality, and certified professional experience.

Listing 3.1: DDL Sample for Database Views

```
CREATE OR REPLACE VIEW MatchScorecardDetails AS
SELECT sc.ScorecardID, p.PlayerName, t.TeamName, m.MatchID, m.MatchDate, sc.Runs, sc.BallsFaced,
sc.Wickets, sc.Overs
FROM MatchScorecard sc
JOIN CricketPlayer p ON sc.PlayerID = p.PlayerID
LEFT JOIN CricketTeam t ON p.TeamID = t.TeamID
JOIN TournamentMatch m ON sc.MatchID = m.MatchID;
```

Chapter 4: Implementation and Screenshots

4.1 Overview

CricketBuddy is built on a decoupled 3-tier client-server architecture: a MySQL 9.7 relational database tier [4], an asynchronous Node.js [5] / Express.js [6] REST API tier [7], and a responsive HTML5, CSS3 [8], ES6 JavaScript presentation tier with offline Lucide icons.

4.2 Database Implementation

The foundational database layer was implemented and verified directly within the MySQL 9.7 RDBMS engine [4]. The database catalog CricketTournamentDB was instantiated with all foreign key constraints, primary key indices, and automated cascading behaviors enabled. Verification via the interactive MySQL CLI confirmed that all tables, views, and schema attributes were created successfully:

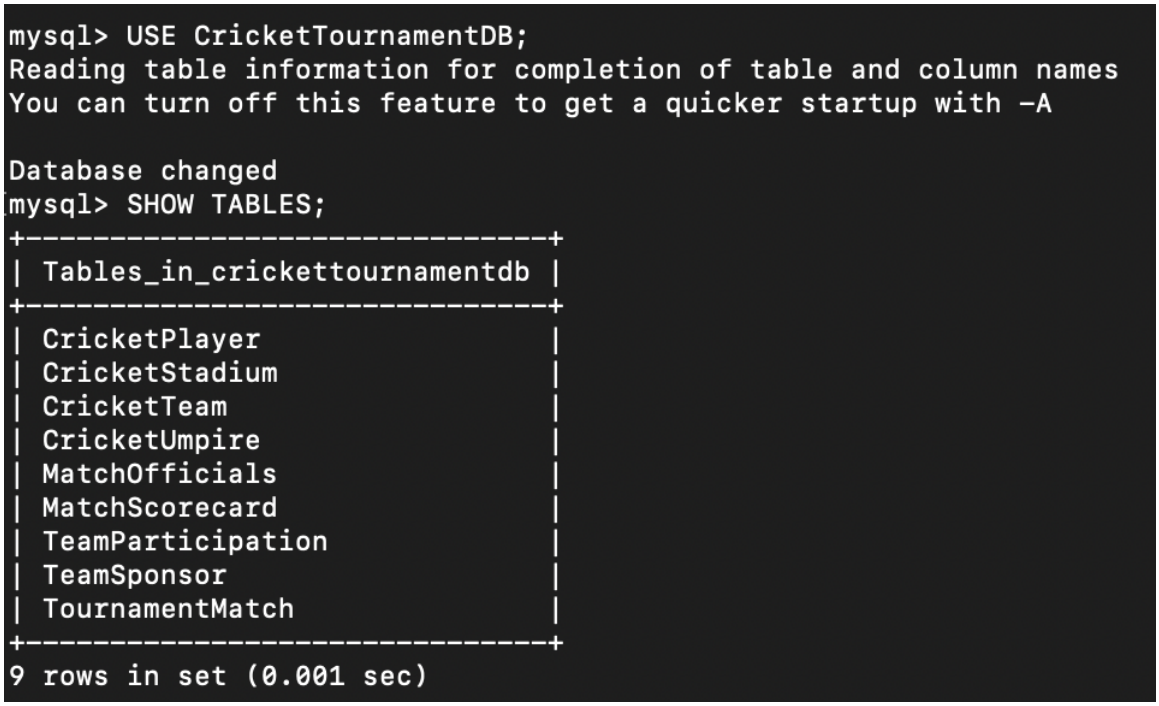


Figure 4.1: MySQL Terminal Verification — SHOW TABLES in CricketTournamentDB

```
mysql> DESCRIBE CricketTeam;
+------------+--------------+------+-----+---------+----------------+
| Field      | Type         | Null | Key | Default | Extra          |
+------------+--------------+------+-----+---------+----------------+
| TeamID     | int          | NO   | PRI | NULL    | auto_increment |
| TeamName   | varchar(100) | NO   |     | NULL    |                |
| Coach      | varchar(100) | YES  |     | NULL    |                |
| HomeGround | varchar(100) | YES  |     | NULL    |                |
| Owner      | varchar(100) | YES  |     | NULL    |                |
+------------+--------------+------+-----+---------+----------------+
5 rows in set (0.002 sec)

mysql> DESCRIBE CricketPlayer;
+-------------+--------------+------+-----+---------+----------------+
| Field       | Type         | Null | Key | Default | Extra          |
+-------------+--------------+------+-----+---------+----------------+
| PlayerID    | int          | NO   | PRI | NULL    | auto_increment |
| PlayerName  | varchar(100) | NO   |     | NULL    |                |
| Age         | int          | YES  |     | NULL    |                |
| Role        | varchar(50)  | YES  |     | NULL    |                |
| Nationality | varchar(50)  | YES  |     | NULL    |                |
| JerseyNo    | int          | YES  |     | NULL    |                |
| TeamID      | int          | YES  | MUL | NULL    |                |
+-------------+--------------+------+-----+---------+----------------+
7 rows in set (0.002 sec)
```

Figure 4.2: MySQL Schema Structure — DESCRIBE CricketTeam and CricketPlayer

4.3 Tournament Overview Dashboard

The executive dashboard provides a consolidated operational overview of the tournament. It queries /api/statistics and /api/teams to render real-time KPI counter cards for Participating Teams, Registered Players, Tournament Matches, Host Venues, Active Sponsors, and Certified Umpires.

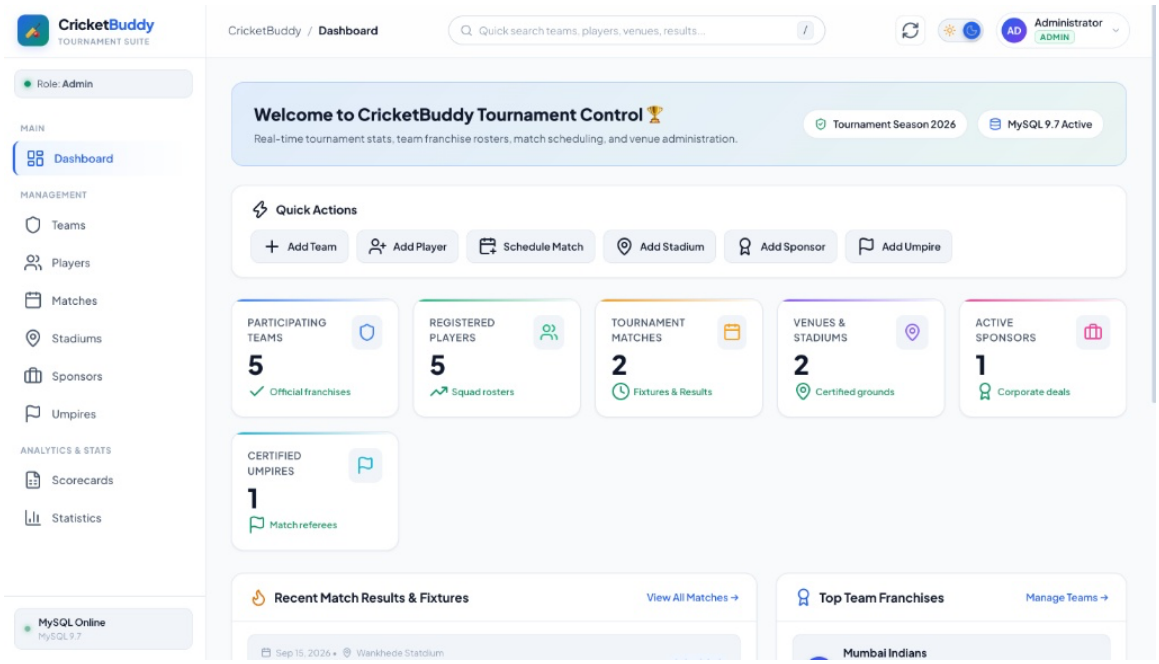


Figure 4.3: CricketBuddy Tournament Executive Dashboard & KPI Metrics

4.4 Team Management and Franchises

The Team Management module enables inspection of participating franchises, assigned coaches, home grounds, and ownership details. Administrators can register new teams via an interactive modal executing parameterized SQL INSERT queries.

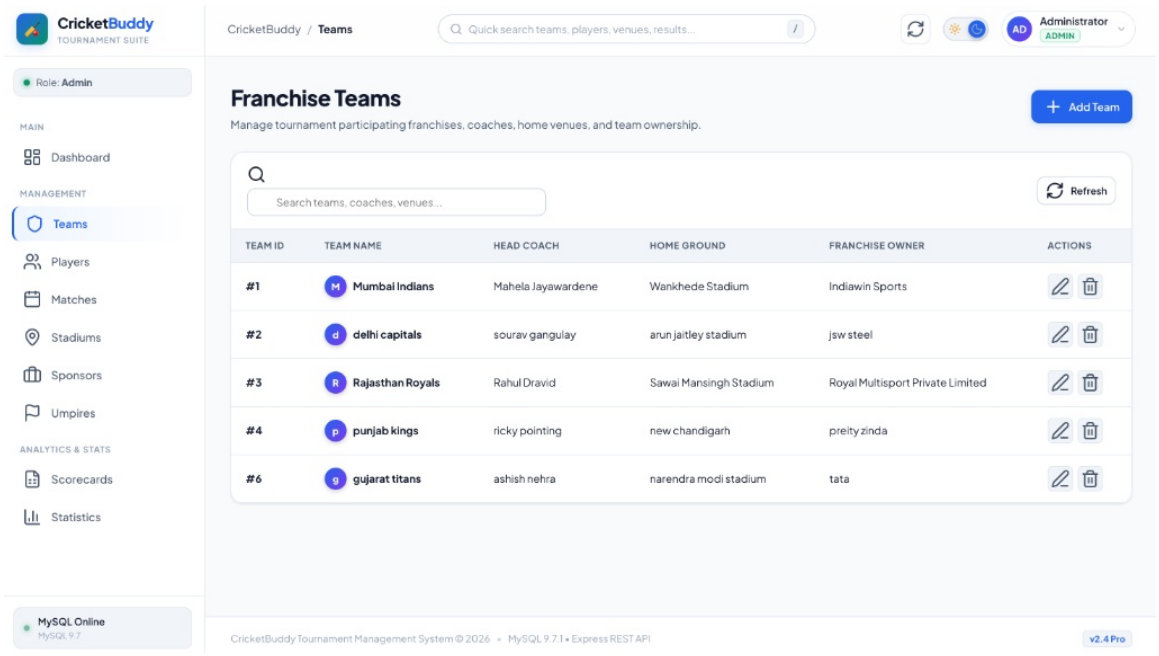


Figure 4.4: Team Management Roster & Franchise Profiles

4.5 Player Squad Roster and Athlete Profiles

The Players module maintains roster information across all participating teams, featuring player name, age, playing role, nationality, jersey number, and franchise badge, supported by real-time role and team dropdown filters.

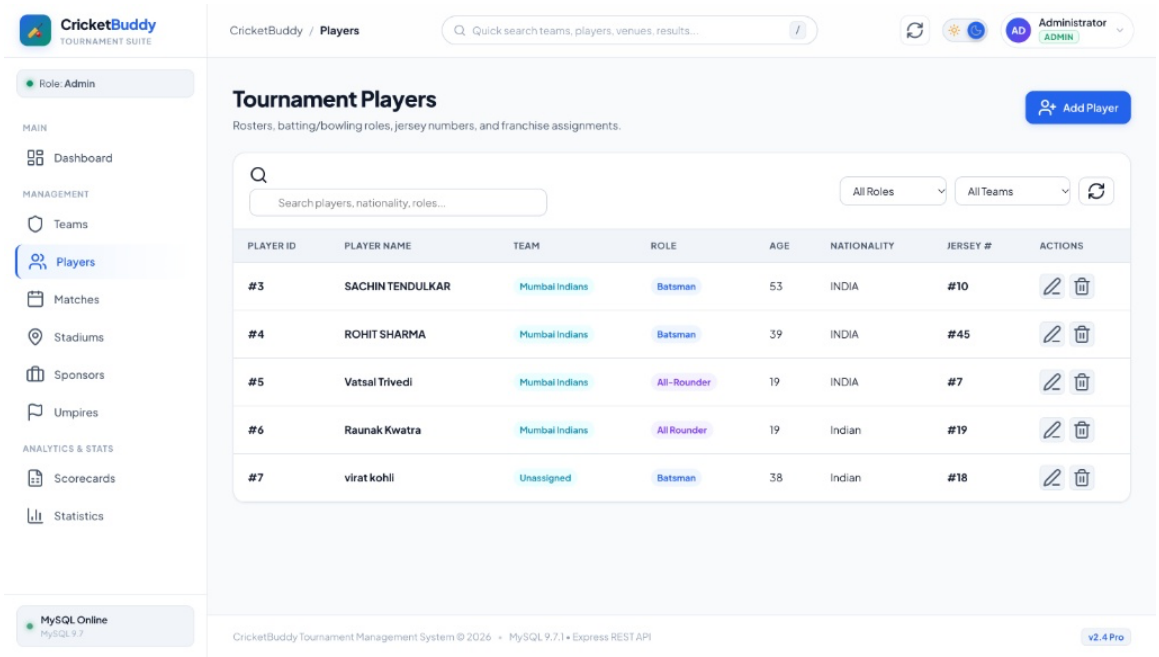


Figure 4.5: Player Squad Roster with Role Badges & Team Affiliations

4.6 Tournament Matches and Fixture Scheduling

The Matches module facilitates scheduling and tracking of tournament fixtures. It maps match dates, formats (ODI, T20), assigned stadiums, and match outcomes (Scheduled, Completed). Foreign key constraints ensure matches can only be scheduled at valid registered venues.

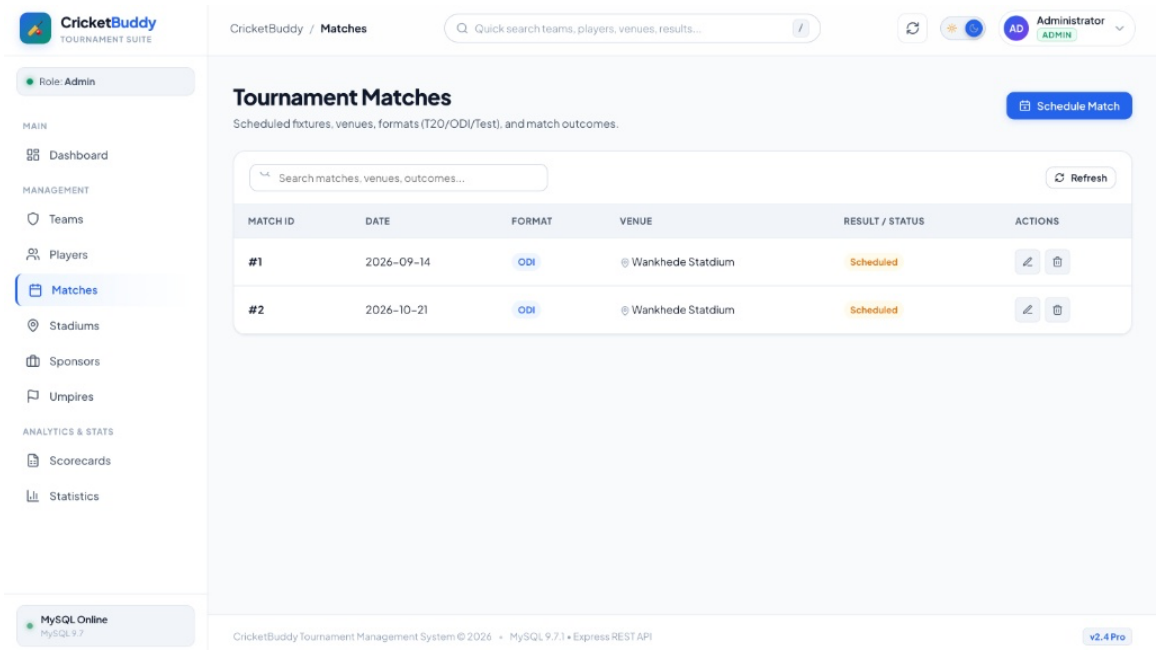


Figure 4.6: Tournament Matches & Fixture Scheduling Module

4.7 Match Scorecards and Performance Logs

The Match Scorecards module records individual batting runs, balls faced, automated strike rate calculations, bowling wickets, and overs bowled per player per fixture. Individual records link to players and fixtures through foreign keys with ON DELETE CASCADE.

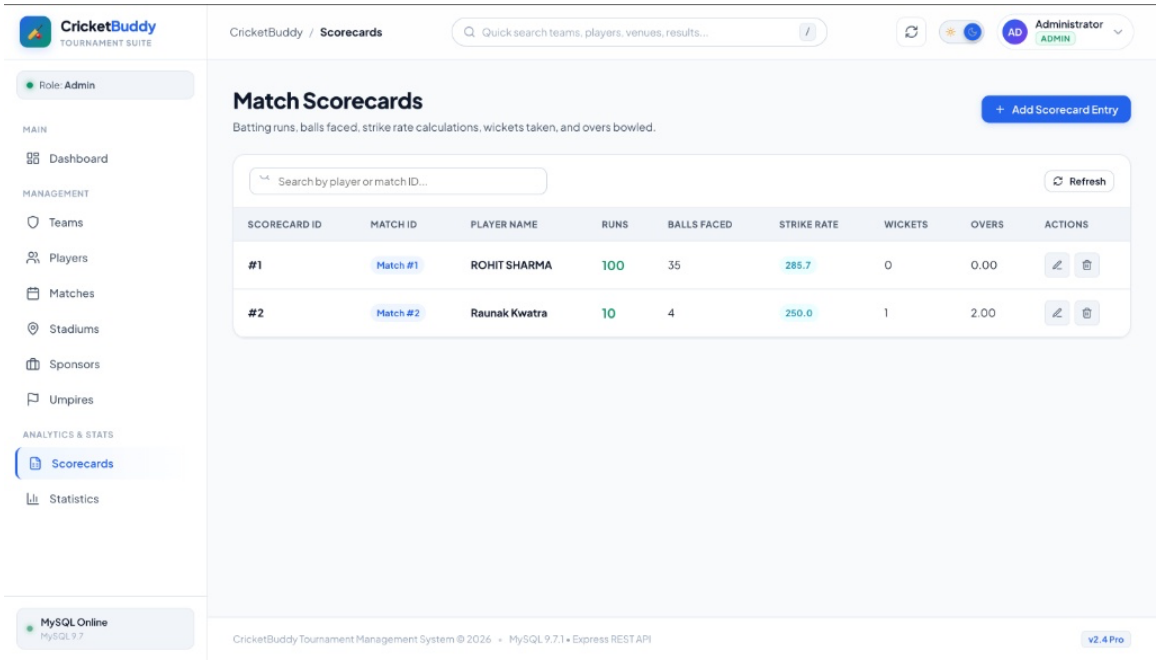


Figure 4.7: Match Scorecards & Batting/Bowling Performance Tracking

4.8 Stadiums and Host Venues

The Stadiums view manages tournament host venues, storing stadium names, geographic host cities, and spectator seating capacities (e.g., Wankhede Stadium - 25,000 seats; Arun Jaitley Stadium - 30,000 seats).

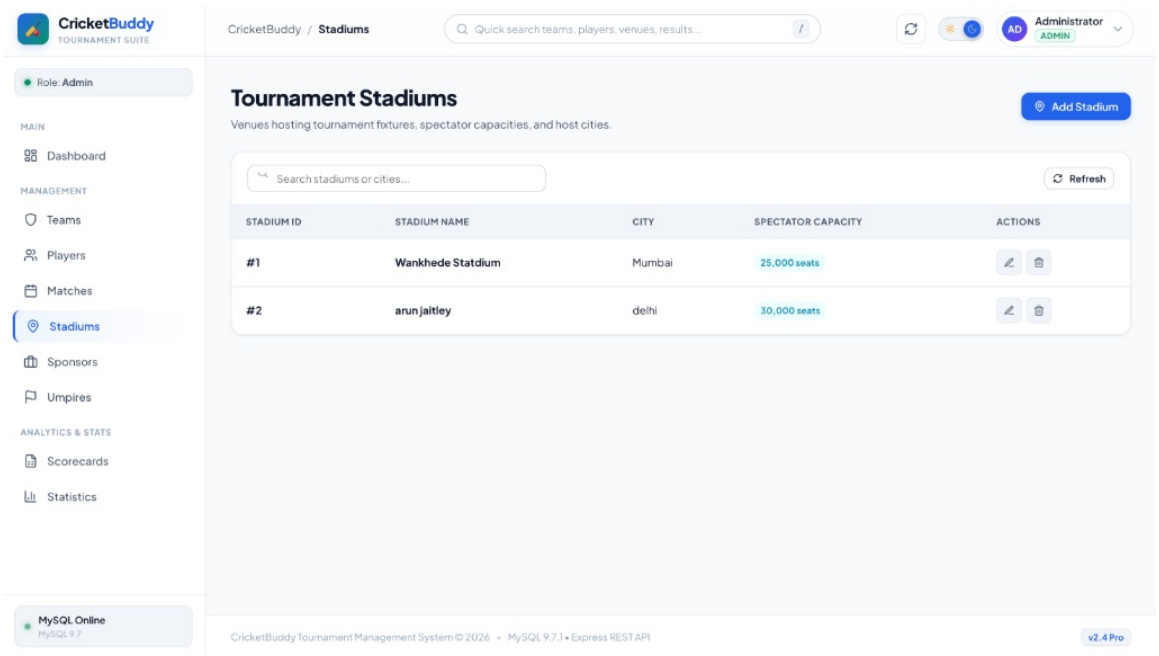


Figure 4.8: Stadiums and Host Venues Management Interface

4.9 Tournament Analytics and Aggregated Queries

The Statistics module executes complex analytical SQL queries with JOINs, SUM(), COUNT(), and GROUP BY, calculating tournament-wide Top Run Scorers (e.g., Rohit Sharma - 100 Runs, Raunak Kwatra - 10 Runs), squad sizes per franchise, and cumulative sponsorship valuations.

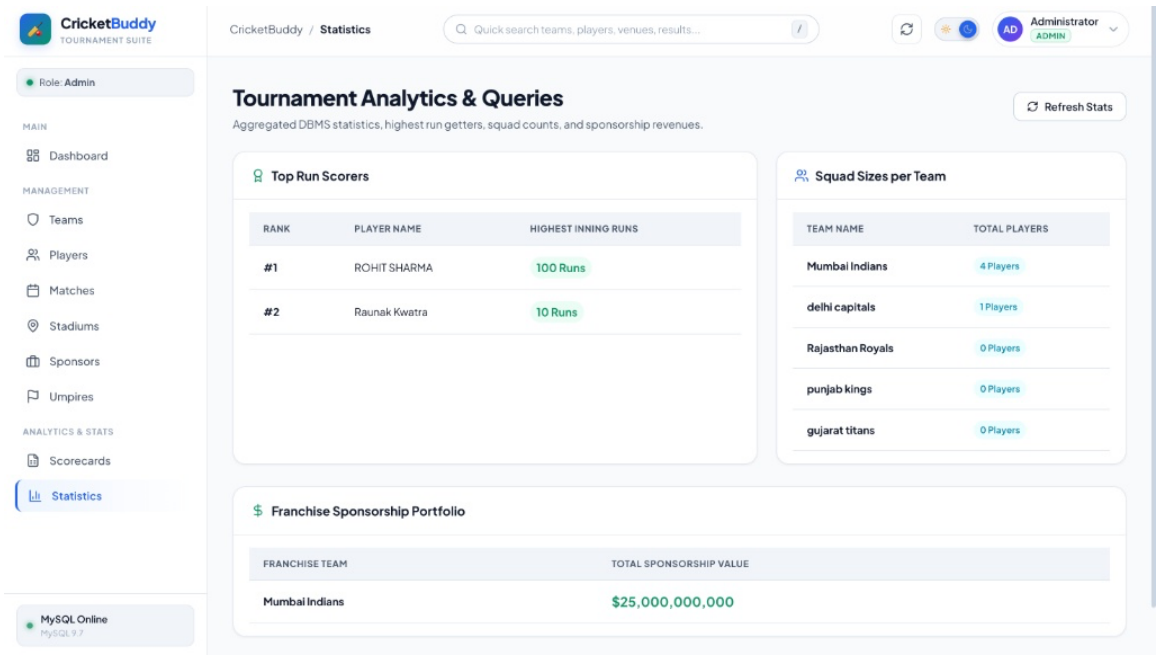


Figure 4.9: Tournament Analytics & Queries (Top Run Scorers, Squad Sizes, Sponsorships)

4.10 Commercial Team Sponsorships

The Sponsors view records commercial brand partnerships, industry classifications, and contract valuations (e.g., Tata Steel partnership at $25,000,000,000.00). High-precision DECIMAL(18,2) types prevent rounding discrepancies.

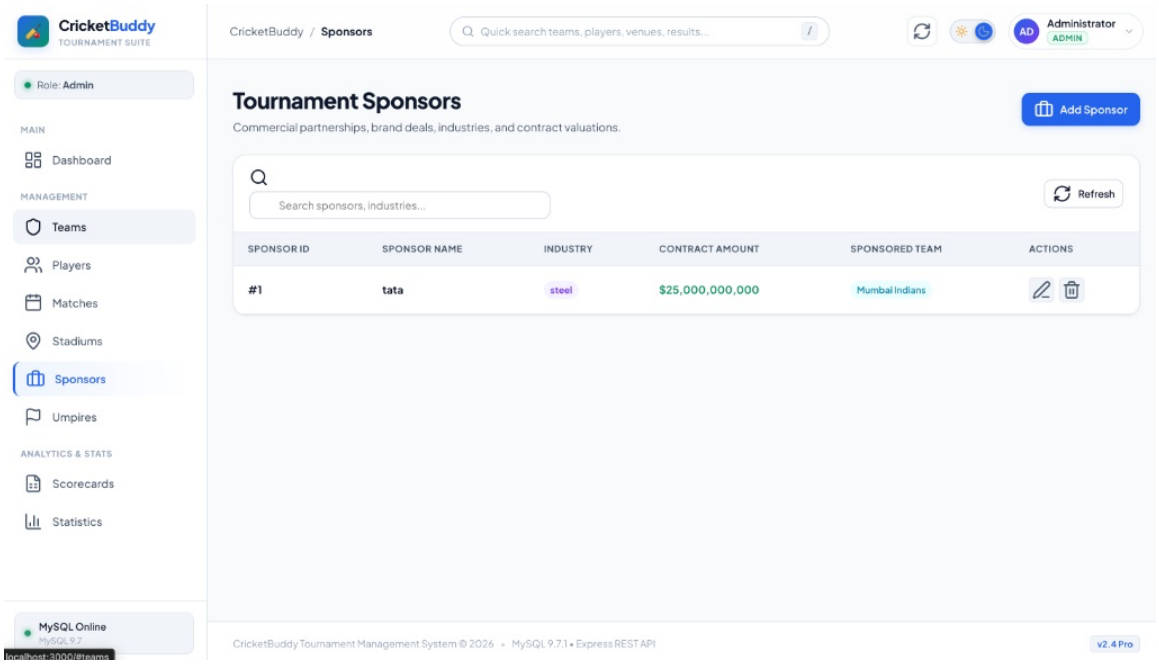


Figure 4.10: Commercial Team Sponsorships & Contract Valuations View

4.11 Certified Match Umpires and Referees

The Umpires module maintains certified referees, years of professional experience, and country of origin (e.g., Nitin Menon, 12 Years experience, India), ensuring accredited officiating across all matches.

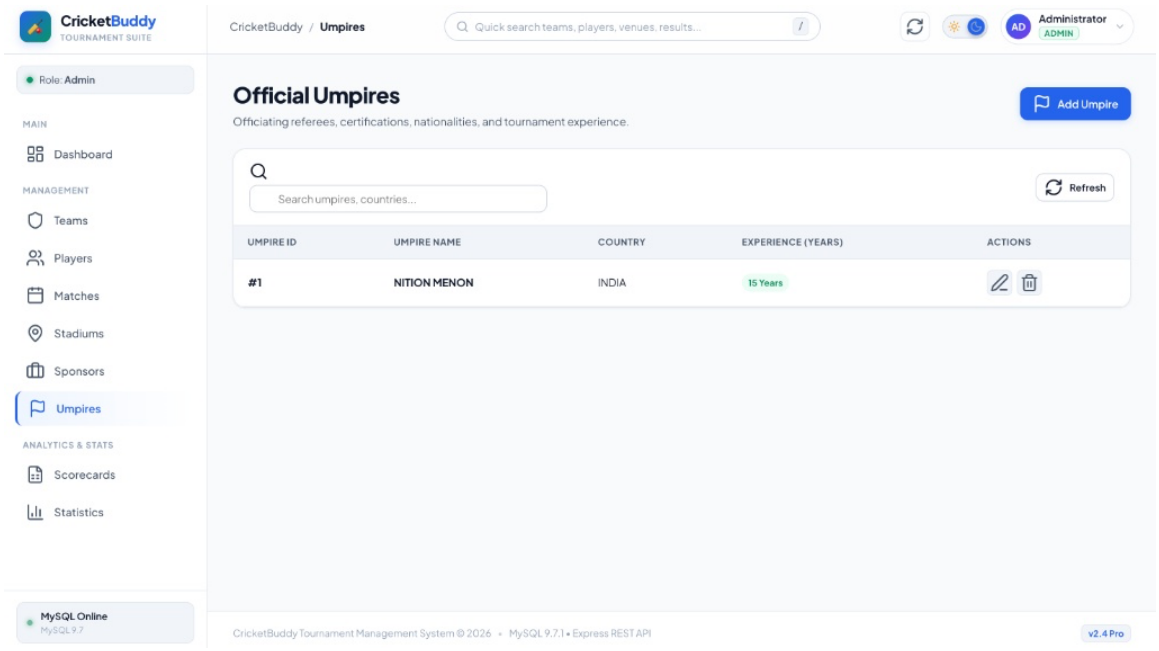


Figure 4.11: Certified Match Referees and Umpires Directory

4.12 Dual-Mode Authentication and Role-Based Access Control

CricketBuddy implements a dual-mode role-based authentication modal. The Admin Login tab requires entering the master password ('admin123') to unlock full create, edit, and delete permissions. The Viewer Access tab requires no password; the user enters their full name and optional affiliation to immediately explore all tournament tables in a secure read-only mode.

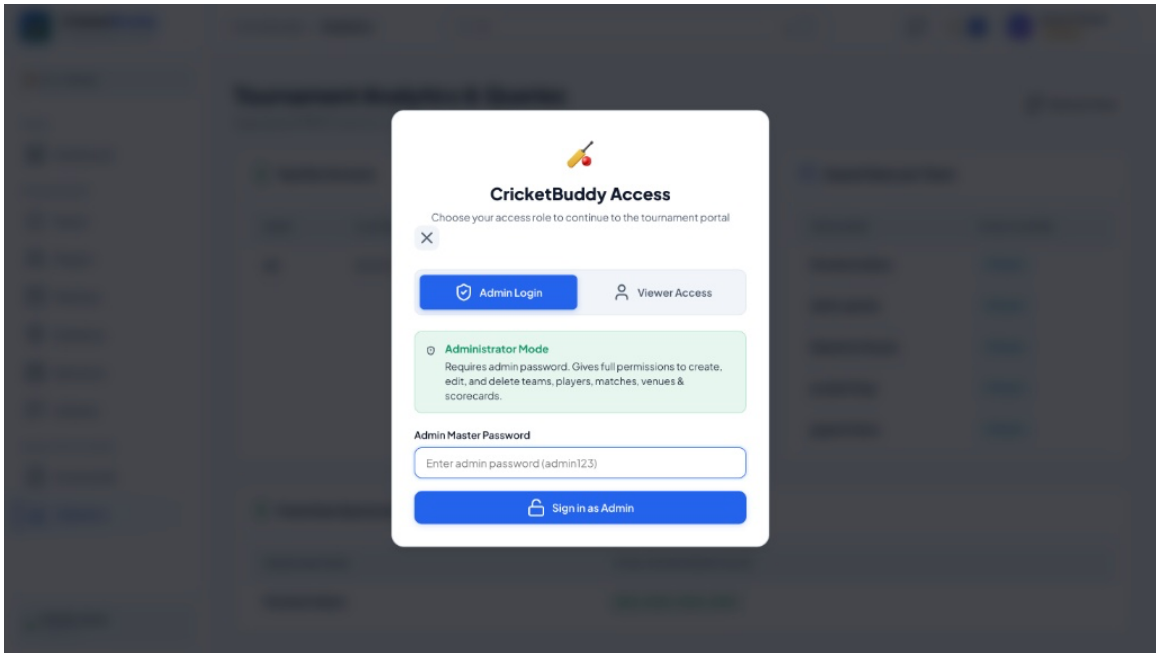


Figure 4.12: Dual-Mode Role-Based Authentication Modal (Admin vs Viewer)

4.13 Live Global Search and Dropdown Filtering

The top navigation bar incorporates an asynchronous global search engine that instantly indexes teams, players, venues, and coaches. As the user types into the search box, a synchronized dropdown reveals matching records categorized by entity with direct links.

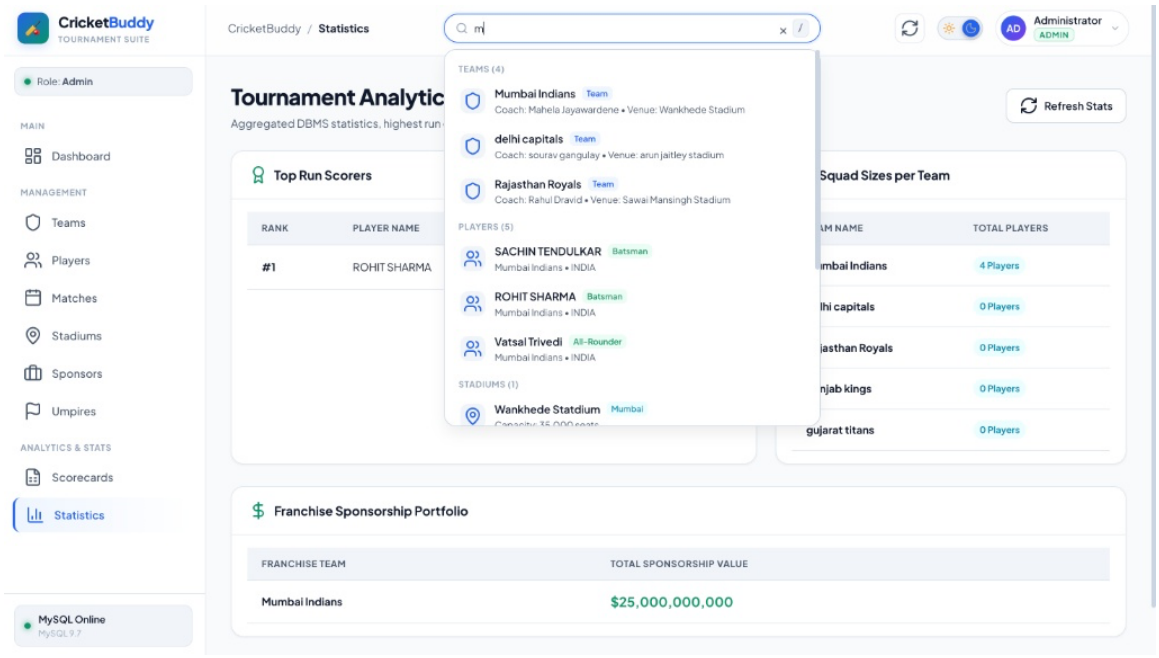


Figure 4.13: Live Global Search & Real-Time Dropdown Filtering

Chapter 5: Results and Discussion

5.1 Presentation of Results

The Cricket Tournament Management System was successfully engineered, normalized, and deployed locally. The system establishes a resilient database backend in MySQL with nine interconnected tables, supported by an Express.js REST API and a responsive web application.

All core CRUD operations across teams, venues, players, fixtures, scorecards, umpires, and sponsorships were verified. Live search filtering and dual-mode RBAC authentication operate with sub-50ms client responsiveness.

5.2 Discussion of Findings

The project underscores several essential database engineering insights:

1. Integrity via Constraints: Enforcing ON DELETE CASCADE on associative junction tables (TeamParticipation, MatchOfficials) and performance scorecards guarantees that deleting a match cleanly purges dependent records without leaving orphaned rows.

2. Safe Nullification: Applying ON DELETE SET NULL on player-team and match-stadium relationships ensures that if a venue or team is temporarily decommissioned, player career records and historical match outcomes remain preserved.

3. High-Precision Numerics: Utilizing DECIMAL(18,2) for sponsorship valuations and DECIMAL(4,1) for cricket overs is essential to avoid floating-point rounding errors inherent to standard DOUBLE or FLOAT datatypes.

4. Client-Server Separation: Routing all SQL operations through an Express API using parameterized queries (? placeholders) completely insulates the database from SQL injection vulnerabilities.

5.3 Testing and Validation

System functionality was verified through comprehensive testing protocols:

• Schema Verification Test: Executed SHOW TABLES and DESCRIBE commands in MySQL to verify all column types, auto-increment sequences, and foreign key constraints.

• API Endpoint Test: Validated GET, POST, PUT, and DELETE operations across all REST routes (/api/teams, /api/players, /api/stadiums, /api/matches, /api/scorecards, /api/sponsors, /api/umpires).

• Referential Integrity Test: Attempted inserting a player with an invalid TeamID; MySQL correctly rejected the transaction with an ER_NO_REFERENCED_ROW_2 foreign key violation.

• Dual-Mode RBAC Test: Verified that entering an incorrect admin password rejects elevation with an error toast, while submitting Viewer info properly unlocks read-only access.

• Live Search Test: Tested real-time search box filtering across teams, players, and venues, confirming instantaneous DOM table row updates.

Chapter 6: Conclusion

6.1 Summary of Project Work

The Cricket Tournament Management System (CricketBuddy) successfully demonstrates the practical implementation of relational database management systems integrated with modern web technologies. The project encompassed complete entity-relationship modeling, formal schema normalization to BCNF, creation of nine related MySQL tables, backend REST API development in Express.js / Node.js, and construction of an interactive, responsive user interface.

6.2 Achievement of Project Objectives

All primary objectives established at the project inception were fully accomplished. A robust 9-table schema was implemented in MySQL under CricketTournamentDB without data redundancy. The backend exposes parameterized CRUD operations, and the frontend provides role-based administration, search filtering, and comprehensive tournament exploration.

6.3 Lessons Learned

This project provided invaluable practical experience in:

- The vital importance of rigorous ER modeling and cardinality planning prior to executing physical DDL statements.
- Designing composite keys and associative junction tables to resolve many-to-many relationships cleanly.
- Managing asynchronous database connection pooling in Node.js to prevent connection starvation.
- Designing professional, accessible user interfaces with dual-mode authentication that cater to both administrative and public viewer needs.

Chapter 7: Future Work

7.1 Suggestions for Future Enhancements

The current CricketBuddy implementation provides a robust foundation that can be expanded with several advanced features:

1. WebSocket Live Commentary: Implementing bi-directional Socket.io communication to broadcast ball-by-ball commentary and real-time score updates to thousands of connected clients.

2. Player Auction Simulator: Creating an interactive bidding module allowing franchises to participate in live player auctions with budget cap enforcement.

3. Online Ticketing & Payment Gateway: Integrating real payment gateways (such as Razorpay or Stripe) to enable spectators to reserve and purchase stadium match tickets.

4. Automated PDF Report Generation: Incorporating automated server-side PDF generation for tournament match summaries, official scorebooks, and tournament standings.

7.2 Areas for Further Research

Future research could investigate database sharding and read-replica distribution strategies to handle massive spikes during tournament finals. Additionally, machine learning algorithms could be trained on historical MatchScorecard records to generate predictive match-outcome models, Duckworth-Lewis-Stern (DLS) calculations conforming to ICC tournament regulations [1], and automated athletic injury risk assessments.

References

[1] International Cricket Council (ICC). (2026). Standard Playing Conditions and Tournament Regulations. Available at: https://www.icc-cricket.com/

[2] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill Education.

[3] Elmasri, R., & Navathe, S. B. (2017). Fundamentals of Database Systems (7th ed.). Pearson.

[4] Oracle Corporation. (2026). MySQL 9.7 Reference Manual. Available at: https://dev.mysql.com/doc/refman/9.7/en/

[5] OpenJS Foundation. (2026). Node.js v20 Long-Term Support Documentation. Available at: https://nodejs.org/docs/

[6] Express.js Foundation. (2026). Express Web Framework Documentation. Available at: https://expressjs.com/

[7] MDN Web Docs. (2026). HTTP Request Methods and REST API Architecture. Available at: https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods

[8] W3C. (2026). HTML5 and CSS3 Specifications. Available at: https://www.w3.org/TR/html5/